

OmniWeave 与代码图谱/代码库记忆生态 深度调研

我们真正的优势、边界，以及可借鉴路线

研究日期：2026-06-23

研究目录：/Users/liuzaoqu/Desktop/develop/sogen/OmniWeave/
research/2026-06-23-codegraph-ecosystem

证据方式	源码 clone、关键文件逐行阅读、官方文档与论文交叉验证
研究对象	26 个项目/系统，18 个本地源码快照，8 类官方/论文资料
输出目标	帮 OmniWeave 做产品与架构决策，而不是项目百科

本报告所有性能/覆盖数字均按来源分级处理：本地实测、项目自述、论文声明、官方文档声明严格分开；
未复现实测的外部指标不作为 OmniWeave 决策事实。

Contents

1	结论先行	3
1.1	一句话结论	3
1.2	OmniWeave 当前最强的五个点	3
1.3	最大的战略边界	3
2	研究方法 with 证据等级	4
2.1	本次调研范围	4
2.2	证据等级	4
2.3	反营销处理原则	4
3	生态分层：这类项目到底在竞争什么	4
3.1	四个真正的问题	4
3.2	生态分类表	5
4	逐项目源码深读	5
4.1	codegraph: OmniWeave 的直接基座，真正价值在 agent 输出面	5
4.1.1	定位	5
4.1.2	关键实现	5
4.1.3	OmniWeave 可借鉴	6
4.2	Codebase-Memory: 宏大、工程野心强，但需要防止过载	6
4.2.1	定位	6
4.2.2	关键实现	6
4.2.3	OmniWeave 可借鉴	7
4.2.4	不应照搬	7
4.3	CodeGraphContext/CGC: 功能广，但复杂度已经明显外溢	7
4.3.1	定位	7
4.3.2	关键实现	7
4.3.3	OmniWeave 可借鉴	7
4.3.4	不应照搬	8
4.4	Serena: LSP/IDE 能力层，不是持久图竞争者	8
4.4.1	定位	8
4.4.2	关键实现	8
4.4.3	OmniWeave 可借鉴	8
4.5	Aider repo-map: 排名/压缩强，但没有可遍历关系	8
4.5.1	定位	8
4.5.2	关键实现	9
4.5.3	OmniWeave 可借鉴	9
4.6	Codanna: Rust/Tantivy 符号索引，agent 消歧协议值得学	9
4.6.1	定位	9
4.6.2	关键实现	9
4.6.3	OmniWeave 可借鉴	9
4.7	Claude Context 与 semantic-search-mcp: 语义入口，不是结构真相	10
4.7.1	Claude Context	10
4.7.2	semantic-search-mcp	10
4.7.3	OmniWeave 可借鉴	10
4.8	Blarify: LSP/SCIP + 图数据库 + LLM 文档生成，适合做反面边界	10
4.8.1	定位	10
4.8.2	关键实现	11
4.8.3	OmniWeave 可借鉴与应避免	11
4.9	code-review-graph: 垂直场景里的好产品化	11

4.9.1	定位	11
4.9.2	关键实现	11
4.9.3	OmniWeave 可借鉴	11
4.10	code-graph-mcp 变体: 同名项目很多, 但参考价值有限	12
4.11	Graphify 与 Understand Anything: 可视化学习层, 而不是核心图引擎	12
4.12	SCIP/Sourcegraph: 精确索引的事实格式, 不是查询引擎本身	12
4.12.1	定位	12
4.12.2	关键设计	12
4.12.3	OmniWeave 可借鉴	13
4.13	Kythe/Glean/CodeQL/Joern/Semgrep/OpenGrep: 重型工业系统的启示	13
4.14	RepoGraph 与 CodeRAG-Bench: 研究评测提醒我们别只测检索	13
5	横向比较	13
5.1	事实来源: Tree-sitter、LSP、SCIP、embedding 的信任边界	13
5.2	存储与查询	14
5.3	增量同步	14
5.4	MCP 工具面	14
6	OmniWeave 的优势: 不是“大而全”, 而是边界可信	15
6.1	优势一: 抓住 LSP/grep 共同盲区	15
6.2	优势二: 诚实边界比高召回更稀缺	15
6.3	优势三: A/B 证明了真实价值边界	15
6.4	优势四: 本地、轻量、可分发	16
7	OmniWeave 应该借鉴什么	16
7.1	优先级 P0: 输出面与工具路由继续压实	16
7.2	优先级 P1: snapshot/artifact 共享	16
7.3	优先级 P1: 可选 SCIP importer	16
7.4	优先级 P2: semantic sidecar	17
7.5	优先级 P2: review vertical skill	17
8	OmniWeave 不应该借鉴什么	17
8.1	不要默认追求大工具面	17
8.2	不要引入多数据库后端	17
8.3	不要把 LLM 文档生成塞进核心	17
8.4	不要把语义搜索当结构事实	17
8.5	不要复制安全分析平台	17
9	建议路线图	18
9.1	Phase A: 输出与工具面硬化	18
9.2	Phase B: snapshot artifact	18
9.3	Phase C: SCIP importer	18
9.4	Phase D: semantic sidecar	18
9.5	Phase E: review vertical skill	18
10	最终判断	19
A	本地源码快照	19
B	关键证据索引	20
C	外部资料与链接	21

1 结论先行

1.1 一句话结论

OmniWeave 的核心优势不在“支持更多语言”、不在“有一个图数据库”、也不在“比 grep 更聪明”。真正的优势是：

它把 **coding agent** 最容易付出高成本、且 **LSP/grep** 结构性看不见的跨边界关系，做成了可追踪、可置信、可预算控制的本地图。

这类项目的生态正在分成三条路：第一条是 结构图/代码智能后端，代表是 codegraph、Codebase-Memory、CGC、Codanna、code-review-graph；第二条是 语义记忆/向量检索，代表是 Claude Context、semantic-search-mcp、Cursor 类索引；第三条是 工业级精确代码索引/安全分析，代表是 SCIP/Sourcegraph、Kythe、Glean、CodeQL、Joern、Semgrep/OpenGrep。OmniWeave 应该站在第一条，但吸收第二条作为入口、第三条作为可选事实来源，而不能被任何一条吞掉。

1.2 OmniWeave 当前最强的五个点

1. 边界关系是第一公民。OmniWeave 明确抓 Python/JS/TS/Go/workflow 到本地脚本、R S4 dispatch、Snakemake/Nextflow artifact DAG、external tool invocation。这不是“多语言列表”，而是 LSP 和 grep 的盲区建模。README 将这一点写得非常清楚：cross-language、cross-process、dynamic dispatch、workflow DAG、external-tool graph 是产品主线，见 README.md:22–31, 135–164。
2. 诚实边界强。generalCrossLangEdges 对动态路径、插值 basename、非真实 subprocess 调用做 gate；推断边带 provenance、metadata.confidence、registeredAt，静态不可解就跳过，见 src/resolution/callback-synthesizer.ts:1941–2039。这比“什么都连上”的图更适合 agent，因为错边比漏边更危险。
3. 价值经过 **agent A/B** 约束。OmniWeave 的 README 和 eval 资料没有把工具包装成“更正确”，而是承认正确性常常与 grep/LSP 打平，优势主要在 reverse/blast-radius 的工具调用、token、延迟和稳定性，见 README.md:35–98 和 eval-results/agent-ab-2026-06-13/。
4. 架构复杂度克制。当前主链是 WASM tree-sitter extraction → SQLite/FTS → resolution/synthesizer → MCP。没有默认外部向量库、没有多后端图数据库、没有把 LLM 文档生成塞进核心。CHECKPOINT 的模块边界和 skip-over-guess 原则与此一致，见 CHECKPOINT.md:38–55, 77–88。
5. **agent** 输出面被当作产品，而不是 API 附属物。继承自 codegraph 的输出预算、explore/node 作为 primary tool、工具 surface 收缩等思想，使图不是“给人查的数据库”，而是“给 agent 消费的行动上下文”。codegraph 的 src/mcp/tools.ts 已经把默认工具压到 explore，OmniWeave README 当前默认五个核心工具，方向一致。

1.3 最大的战略边界

OmniWeave 不应试图成为所有东西：

- 不应替代 LSP 在同语言精确导航上的优势。LSP/SCIP 有编译器语义，OmniWeave 的价值在跨边界和零配置。
- 不应用向量检索伪装成结构事实。语义搜索适合“auth 在哪儿”这种概念入口，但不能直接产生 calls/crossLang/overrides 这类可置信边。
- 不应追求 Codebase-Memory/CGC 式大工具面。14 个、20 个、甚至更多 MCP tools 会提高试错概率；OmniWeave 更应该把少数工具做到足够完整。
- 不应为了“企业感”引入多图数据库、多 UI、LLM 文档生成、自然语言 Cypher 等核心依赖。那些是外围产品层，不是最小可信 substrate。

2 研究方法与证据等级

2.1 本次调研范围

本次建立了独立研究目录：

```
/Users/liuzaoqu/Desktop/develop/sogen/OmniWeave/research/  
2026-06-23-codegraph-ecosystem
```

其中 metadata/repos.tsv 记录 26 个项目，metadata/repo-snapshot.tsv 记录 18 个本地源码快照的 commit、文件数、体量；project-inventory.md 记录证据等级。源码 clone 不是为了“多看几个 README”，而是为了验证每类系统真正的架构重心。

2.2 证据等级

等级	含义
source-clone	本地 clone 并阅读关键实现。本文可讨论源码结构、工具面、存储、索引管线、输出协议。
paper+source-clone	同时具备论文/项目叙述和可读源码；本文区分论文目标与实现质量。
source-sampled	没有完整 clone 或只取官方/代表性文件；仅讨论公开设计方向，不做实现细节断言。
official-docs	官方文档/博客/产品说明；可用于定位与能力边界，但数字按“官方声明”处理。
paper	论文指标或 benchmark；除非本地复现，否则不作为 OmniWeave 事实，只作为方向信号。

2.3 反营销处理原则

这类项目普遍喜欢宣称“百万行秒级索引”“数百倍 token 节省”“支持上百语言”。本报告采用三条过滤规则：

- 项目自述数字只标注来源，不直接变成结论。例如 Codebase-Memory 的 Linux kernel 级索引数字、Codebase-Memory paper 的 83% answer quality、code-review-graph 的 82x/528x token reduction，都按来源分级。
- 语言数量不是主要指标。关键是该语言是否有可用关系、是否能跨文件/跨进程、是否有 false-positive gate、是否能被 agent 直接消费。
- 工具数量不是优势。MCP 工具越多，agent 选择成本越高。真正优势是少数工具是否能一次返回可行动上下文。

3 生态分层：这类项目到底在竞争什么

3.1 四个真正的问题

所有“code graph / codebase memory / code intelligence for agents”项目，本质上都在回答四个问题：

- 如何把代码变成可查询的事实？选择 Tree-sitter、LSP、SCIP、compiler indexer、regex、embedding，决定了事实的可信度。
- 如何把事实保持新鲜？选择 file watcher、git diff、hash/Merkle、incremental rebuild、snapshot artifact，决定了开发时可用性。

3. 如何把事实交给 **agent**? 选择 MCP tools、skills、slash commands、CLI、IDE extension、query language，决定了 agent 会不会用错。
4. 如何证明它值得存在? 选择 unit eval、agent A/B、retrieval benchmark、security finding、token accounting，决定了项目是否会陷入自我宣传。

3.2 生态分类表

类型	代表项目	真正强项	对 OmniWeave 的意义
Agent-first 结构图	codegraph、Omni-Weave、code-review-graph、Codanna	少数查询直接给 agent 行动上下文；强调 token/tool-call 经济性	主战场。继续压低形态税、提高输出充分性。
大型 codebase memory	Codebase-Memory、CGC	多语言、大工具面、持久 DB、artifact、query language、安装面	借鉴 artifact/安全查询/控制面；避免工具面膨胀。
语义检索/向量记忆	Claude Context、semantic-search-mcp、Cursor indexing	概念召回、自然语言入口、跨大代码库模糊搜索	做可选 sidecar：只产生候选 seed，不产生结构真边。
LSP/IDE 能力层	Serena、Blarify LSP path	find references、rename、diagnostics、precise edits	做可选 validation/edit companion，不替代 core graph。
工业精确索引	SCIP/Sourcegraph、Kythe、Glean	编译器级导航、跨 repo symbol、事实格式、DataLog/Angle 查询	借鉴事实格式和 importer；不把核心变成重型索引平台。
安全/程序分析图	CodeQL、Joern、Semgrep/OpenGrep	variant analysis、CPG、taint/rule matching	可做未来安全插件，不是 coding-agent 日常上下文核心。
研究型 RAG benchmark	RepoGraph、CodeRAG-Bench、CodexGraph/-CodeRAG	衡量检索对生成/修 bug 的影响	借鉴评测问题设计，不照搬实现。
可视化学习工具	Graphify、Understand Anything	onboarding、dashboard、文档/知识图、团队理解	可做外围报告/学习层；不要污染 MCP 核心。

4 逐项目源码深读

4.1 codegraph：OmniWeave 的直接基座，真正价值在 agent 输出面

4.1.1 定位

codegraph 不是最炫的图数据库项目，但它非常适合 coding agent：轻量、本地、Tree-sitter/WASM、SQLite、MCP，且工具设计围绕 agent 的上下文预算。OmniWeave 最初继承它，是正确路线。

4.1.2 关键实现

本地快照：colbymchenry/codegraph@a893156，见 metadata/repo-snapshot.tsv。

- 工具 **surface** 极度收缩。src/mcp/tools.ts 中 getStaticTools 默认只暴露 codegraph_explore，其他工具通过环境 allowlist 才出现。这背后是非常强的 agent 产品判断：让模型少选错工具。

- 输出预算是核心逻辑。tools.ts 有 inline output cap、adaptive budget、small repo 下减少 relationships/additional files、test file filtering 等策略。它把“别炸上下文”当成主逻辑，而不是响应末尾的截断。
- 解析热路径有安全阀。src/extraction/index.ts 中有 parse timeout、worker thread、按文件数 recycle worker、batch IO、watch stale banner。WASM 内存无法 shrink 这类现实问题被纳入设计。
- watcher 和 indexer scope 对齐。src/sync/watcher.ts 对平台 watcher 复杂度、debounce、ignore matcher 复用、inotify/EMFILE degrade 都有处理。
- callback synthesizer 走“高精度低召回”。src/resolution/callback-synthesizer.ts 的动态边合成带 fan-out cap 和 provenance，而不是任意联想。

4.1.3 OmniWeave 可借鉴

codegraph 已经证明：**agent-first** 的图不是“查询能力越多越好”，而是“一个 **primary query** 能不能把探索压成一跳”。OmniWeave 应继续强化 explore/node，而不是把所有边种都暴露成独立工具。

建议：

1. 将默认工具面进一步 A/B：是否可以默认只给 explore、node、search，把 callers/impact 作为 explore 内部路线或 allowlist。
2. 把输出预算策略从“字符上限”推进到“任务形状预算”：single-point、reverse/blast、flow survey、file read replacement 各自有不同输出模板。
3. 对所有新 edge kind 先问：它是否能显著减少 agent 工具调用？如果不能，就不要进默认输出。

4.2 Codebase-Memory：宏大、工程野心强，但需要防止过载

4.2.1 定位

Codebase-Memory 是本轮最接近“代码库记忆后端”的系统：Tree-sitter 多语言、LSP hybrid、SQLite、FTS、Cypher-like 查询、MCP 14 tools、artifact 持久化、安装到多个 agent 的控制面。它的产品叙事很强：让 agent 不再反复重读整个 repo。

本地快照：DeusData/codebase-memory-mcp@684e35b。

4.2.2 关键实现

- 14 个 MCP 工具。src/mcp/mcp.c:280-448 定义 index_repository、search_graph、query_graph、trace_path、get_architecture、ingest_traces、ADR 等工具。这体现了“平台化”路线。
- SQLite authorizer 值得重视。src/store/store.c:540-581 注册 SQLite authorizer，拒绝 ATTACH/DETACH 等危险操作。对于任何暴露 query language 的图服务，这是底线。
- RAM-first pipeline。src/pipeline/pipeline.c:1-12,621-754 把 discover/structure/source/LZ4/definition/resolve/post-pass/dump 串成多阶段流程，先在内存图里做 pass，再 dump 到 SQLite。
- 自定义 SQLite 函数。store.c 注册 REGEXP/IREGEXP、camel-case splitter、int8 cosine，FTS5 采用 contentless 表。这是把查询性能推入存储层的做法。
- 语义层很激进。src/semantic/semantic.c 使用 TF-IDF/RRI/API/type/decorator/dataflow/graph diffusion 等信号，不一定读取源码，也不等同于 embedding API。这条路线可以做概念召回，但必须避免变成结构真相。

- **watcher** 是 **git-based**。src/watcher/watcher.c 走 git status/HEAD tracking，而不是 fsnotify；优点是跨平台和版本语义，缺点是对未纳入 git 的临时文件语义更弱。

4.2.3 OmniWeave 可借鉴

- **artifact** 共享。Codebase-Memory 的 .codebase-memory/graph.db.zst 思路值得借鉴：OmniWeave 可以有 schema-versioned、hash-verified、read-only importable snapshot，用于 CI/团队冷启动。
- **安全查询层**。如果 OmniWeave 未来支持 SQL/Cypher-like 查询，必须先有 authorizer、只读 AST、复杂度限制、row cap、timeout。
- **安装/控制面**。Codebase-Memory 和 code-review-graph 都在自动安装 MCP/skills/instructions 上用力。OmniWeave 的 installer 可以吸收“多 agent 目标 + dry-run + sibling config preservation”。

4.2.4 不应照搬

- 14 tools 不适合作为 OmniWeave 默认形态。工具多会把 agent 重新带回“试错导航”。
- query_graph 类自由查询若没有安全和输出治理，会把复杂度转嫁给 agent。
- ingest_traces 这类运行时边如果没有验证闭环，容易制造“看起来很强、实际上不可维护”的边。

4.3 CodeGraphContext/CGC：功能广，但复杂度已经明显外溢

4.3.1 定位

CGC 把 code graph 做成多后端、多工具、多语言、bundle、watch、SCIP 可选、Spring/ORM/MyBatis 等 framework pass 的平台。它有很强的“企业代码理解平台”企图。

本地快照：CodeGraphContext/CodeGraphContext@c235df7。

4.3.2 关键实现

- 工具面很大。tool_definitions.py 包含 indexing、find、relationship analysis、watch、Cypher、package graph、dead code、complexity、visualize、bundle、context switch、Java Spring endpoint/bean/datasource 等。
- **async indexing pipeline**。tools/indexing/pipeline.py 先 discover/pre-scan imports，再 semaphore 并发 parse，最后按确定顺序写图；之后再 do inheritance、CALLS、Spring、Maven/-Gradle、ORM、MyBatis、embeddings 等 post-pass。
- **SCIP optional path**。scip_indexer.py 能本地或 Docker 跑 SCIP indexer，针对 C/C++ 等依赖 compile commands 的语言 fallback 到 Tree-sitter。
- **ROADMAP** 暴露出真实问题。tool definitions 与 handler drift、Kuzu Cypher translation hack、bundle hash/signature 不足、graph builder recursion、16 language query toolkit stub 等，说明平台化会快速制造维护面。

4.3.3 OmniWeave 可借鉴

- 可选 **SCIP adapter**。对 TypeScript/Java/Rust/C++ 等有成熟 SCIP indexer 的语言，OmniWeave 可以 ingest SCIP 作为 provenance=scip 的补充事实，不应把 SCIP 作为必须依赖。
- **bundle/context switch**。大 monorepo 下允许 agent 显式切换 project graph 或 import bundle，有价值；但必须有 hash、schema version、签名/路径 guard。
- **framework pass** 要证据门禁。Spring/ORM/MyBatis 类 pass 可以借鉴，但只有在 false-positive gate 和 eval fixture 存在时才进核心。

4.3.4 不应照搬

CGC 的最大教训是：多数据库后端、多 UI、多 framework、多 query language 不是天然高级，而是维护债。OmniWeave 当前的 SQLite + MCP + CLI 主线更清晰。除非有明确用户收益和 eval，不能引入后端多态。

4.4 Serena: LSP/IDE 能力层，不是持久图竞争者

4.4.1 定位

Serena 是基于 LSP/JetBrains 的 coding-agent 工具层。它擅长 symbol overview、find symbol、find references、implementations、diagnostics、rename/safe delete/insert/replace 等 IDE 操作，也有 project memory。它不是持久图数据库，强项是“精确编辑能力”。

本地快照：oraio/serena@dd7eb6d。

4.4.2 关键实现

- agent.py 里有 ToolSet/AvailableTools、include/exclude/fixed tool set、legacy name mapping、exclude editing tools 等工具模式控制。
- ls_manager.py 并行启动 language servers，按文件选择合适 LS，失败可 restart。
- symbol_tools.py 中 GetSymbolsOverviewTool、FindSymbolTool、FindReferencingSymbolsTool 都强调 compact result、body/depth/include filters、max answer chars。
- editing tools 以 symbol body 为单位替换、插入 before/after、rename、safe delete，路线更接近 IDE refactor。

4.4.3 OmniWeave 可借鉴

Serena 最值得借鉴的是 **tool mode**：read-only、editing、initial instructions、project activation、memory 各自可开关。OmniWeave 未来如果进入编辑/重构闭环，应把 LSP/Serena 类能力作为 companion，而不是让 graph core 承担 rename correctness。

建议：

1. 增加 validation mode：OmniWeave 给出结构候选，LSP/SCIP 验证 same-language references。
2. 对 MCP tools 提供 read-only strict mode 与 edit-assist mode，避免 agent 在纯研究时看到编辑工具。
3. 借鉴 Serena 的 shortened output 与 ambiguity handling，但输出要保持 OmniWeave 的 source + blast radius 形态。

4.5 Aider repo-map：排名/压缩强，但没有可遍历关系

4.5.1 定位

Aider 的 repo-map 是一个上下文压缩/排序系统，不是图数据库。它把 tree-sitter tags、defs/refs、聊天上下文、文件名/identifier mention、PageRank、token budget binary search 组合起来，产生“该给模型看的 repo map”。

本地快照：Aider-AI/aider@5dc9490。

4.5.2 关键实现

- `aider/repomap.py` 缓存 tags, 使用 `tree-sitter query` 文件和 `Pygments fallback`。
- `get\_ranked\_tags` 构建 `networkx.MultiDiGraph`, `defs/refs` 构边, 按聊天文件、提到的文件名、`identifier mention` 个性化 PageRank。
- `identifier` 权重会提高更具体的 `snake/kebab/camel` 名称, 降低 `private/frequent identifiers`。
- 输出通过 `token budget binary search` 调整到 `max\_map\_tokens`。

4.5.3 OmniWeave 可借鉴

OmniWeave 不应复制 `repo-map` 作为替代, 但可以学习两个点:

1. **Personalized ranking**. `explore` 可以把用户 `query`、当前文件、最近打开符号作为 `personalization seed`, 对候选 `symbol/source` 排序。
2. **Token-budget binary search**. 在输出临界区, 用二分寻找最大可包含完整函数/完整文件段, 而不是粗暴截断。

边界: `repo-map` 没有 `callers(x)`、`impact(x)`、跨进程边、`workflow artifact DAG`, 因此不是 OmniWeave 的核心竞品, 而是输出排序参考。

4.6 Codanna: Rust/Tantivy 符号索引, agent 消歧协议值得学

4.6.1 定位

Codanna 是 Rust 写的 `symbol index + relationship graph + semantic sidecar`。它的工程重点在高性能 pipeline、`Tantivy` 搜索、`symbol\_id:N` 直接查找、`MCP/CLI` 双形态、`watch/server lock`。

本地快照: `bartolli/codanna@cb2c095`, 本次使用 `repos/codanna-clean; repos/codanna` 是中断 checkout, 不作为证据。

4.6.2 关键实现

- 五阶段 **pipeline**. `src/indexing/pipeline/mod.rs:1-20,260-543` 明确 `DISCOVER → READ → PARSE → COLLECT → INDEX`。`READ/PARSE` 多线程, `COLLECT` 单线程分配 ID, `INDEX` 单线程写 `Tantivy`, 并用 `bounded channels` 做 `backpressure`。
- **Tantivy schema** 分开 `exact` 与 `ngram`。`src/storage/tantivy.rs:118-153` 同时有 `name exact STRING` 和 `name\_text ngram TEXT`, 避免 `MyService` 匹配 `Main` 这类问题。
- 关系 **metadata**. `relation\_receiver`、`relation\_static\_call` 等字段说明它在 `call resolution` 上处理 `receiver/static call`。
- **symbol_id** 消歧. `src/retrieve.rs:66-86,119-190` 对 `name lookup` 的 `ambiguous case` 会返回 `symbol\_id:N` 建议; `src/mcp/mod.rs:84-100,299-322` 也把 `symbol id` 作为 `MCP` 参数。
- 向量侧车. `src/vector/storage.rs` 用 `mmap` 二进制向量文件, 自称 `<1us access`; 但 `read\_vector` 在片段中线性扫描 `vector id`, 所以该指标需要结合 `segment/index` 设计验证, 不能按注释直接采信。

4.6.3 OmniWeave 可借鉴

- 直接 **ID** 工作流. OmniWeave 的 `node/explore` 输出可以稳定暴露短 ID 或 `continuation key`, 让后续工具无需重新按名字消歧。
- **exact + fuzzy** 双字段. `SQLite FTS` 已有基础, 但 `symbol name` 的 `exact/fuzzy/substring` 语义应明确分层, 避免误召回污染 `agent`。

- **pipeline telemetry**。Codanna 对 stage wait time、wall time、thread count 的度量值得吸收，用于 OmniWeave 的 indexing profile。

4.7 Claude Context 与 semantic-search-mcp：语义入口，不是结构真相

4.7.1 Claude Context

Claude Context 是 Zilliz 的 MCP semantic code search。README 明确需要 OpenAI embedding key 和 Milvus/Zilliz Cloud，目标是让 Claude Code 等 agent 从百万行代码中用语义搜索拿相关片段。它是“外部向量数据库 + embedding + MCP”的产品化路线。

本地快照：zilliztech/claude-context@627eb2b。

关键实现：

- **packages/core/src/context.ts:131-184** 初始化 OpenAI embedding、vector database、AST splitter，默认支持一组扩展和 ignore patterns。
- **Context.getCollectionName** 用 codebase path hash 区分 collection，并 sanitize override，避免多个项目碰撞。
- **packages/mcp/src/handlers.ts:10-23** 用 indexingTasks 跟踪后台 indexing，clear index 时可以 cancel/await，避免“清空后后台继续写入”的竞态。
- **handlers.ts:31-154** 对 snapshot 0/0 completed poisoning 做启动修复，避免错误状态导致无限 reindex。
- **packages/mcp/src/sync.ts:59-180** 有全局 sync lock、stale lock recovery、background sync interval。

这些细节说明 Claude Context 的难点不是“embedding 一下代码”，而是外部向量库状态、snapshot、并发 index、同步锁、路径归一化、collection recovery。若 OmniWeave 引入语义 sidecar，必须把这些状态复杂度隔离在 optional 模块。

4.7.2 semantic-search-mcp

semantic-search-mcp 是更小的语义搜索基线：FastEmbed/Jina code embeddings、本地 SQLite、vector + FTS5 RRF、watcher、pause/resume/reindex/clear。它证明语义检索可以轻量，但仍只回答“哪里可能相关”，不回答“谁调用谁”。

4.7.3 OmniWeave 可借鉴

1. 建一个 optional semantic_seed 工具或 internal stage：自然语言 → candidate symbols/files → 交给 explore 做结构化上下文。
2. 语义结果必须带 source=semantic 和低置信标签，不能被写入 calls/crossLang 等结构边。
3. 不默认依赖外部 API 或 cloud vector DB；如果接入，必须是 plugin/sidecar，不污染 local-first 核心。

4.8 Blarify：LSP/SCIP + 图数据库 + LLM 文档生成，适合做反面边界

4.8.1 定位

Blarify 目标是把本地 repo 表示成图，让 LLM traverses graph 理解逻辑；支持 Tree-sitter hierarchy、LSP/SCIP reference resolver、Neo4j/FalkorDB 管理、workflow/documentation 生成、MCP tools。

本地快照：blarApp/blarify@ada9bbd。

4.8.2 关键实现

- `project\_graph\_creator.py`: 69–72 先 create code hierarchy, 再 create relationships from references。
- `project\_graph\_creator.py`: 170–246 批量对所有 definition nodes 查 references, 再用 relationship creator 建边。这是 LSP/SCIP 精确引用路线。
- `hybrid\_resolver.py`: 20–35, 71–185 提供 SCIP_ONLY/LSP_ONLY/SCIP_WITH_LSP_FALLBACK Python/TypeScript 走 SCIP, 否则 LSP。
- `prebuilt/graph\_builder.py`: 64–136 把 graph build、DB save、workflow discovery、documentation generation、embedding 绑在一个 builder 上。

4.8.3 OmniWeave 可借鉴与应避免

可借鉴: SCIP/LSP 作为 reference resolution adapter; workflow discovery 作为后处理 pass 的思路; GraphEnvironment 这类环境 metadata。

应避免: 把 LLM documentation、workflow generation、embedding、graph DB persistence 都塞进核心 builder。OmniWeave 的核心必须保持“静态事实 → 可信边 → agent 输出”, 其余都应是插件。

4.9 code-review-graph: 垂直场景里的好产品化

4.9.1 定位

code-review-graph 专注 PR/代码审查: Tree-sitter 结构图、SQLite、blast radius、风险评分、CI sticky comment、skills/slash commands。它不是通用 substrate, 但在“review changes”场景里非常完整。

本地快照: [tirth8205/code-review-graph@b72413c](https://github.com/tirth8205/code-review-graph)。

4.9.2 关键实现

- `README.md`: 171–222 很认真地区分 benchmark: 528x 是最大值, 不是典型值; recall 1.0 是 graph-derived upper bound, 不是独立正确性。
- `docs/REPRODUCING.md`: 10–60 校准 chars/4 token estimate, 并提供 tiktoken verify; `docs/REPRODUCING.md`: 100–128 给出完整 eval 命令和 failure semantics。
- `code\_review\_graph/graph.py`: 32–79 用 SQLite 存 nodes/edges/metadata, edge 上有 confidence/confidence_tier。
- `code\_review\_graph/tools/review.py`: 24–186 `get\_review\_context` 根据 changed files 算 impact radius、source snippets、review guidance 和 context savings。
- `parser.py`: 84–146 支持大量扩展, 包括 R、Perl、Lua、Objective-C、Bash、Jupyter、SQL、Vue/Svelte 等; 但很多语言是浅结构抽取。

4.9.3 OmniWeave 可借鉴

- 诚实 **benchmark** 写法。把最好情况、典型情况、circular upper bound、co-change independent-ish evidence 分开。这一点和 OmniWeave README 当前风格一致, 应该延续。
- **review vertical package**。OmniWeave 可以提供 `review-delta` skill/command, 但底层仍用 `impact/explore`, 不要把 review 特化逻辑塞进 core graph。
- **context savings metadata**。对每次 MCP 输出估算“替代 grep/read 节省了什么”, 有助于 agent 和用户理解工具价值。

4.10 code-graph-mcp 变体：同名项目很多，但参考价值有限

本轮看了两个同名/近名项目：

- **entrepreneur4lyf/code-graph-mcp@374d41e**: Python、ast-grep/rustworkx、多语言、usage guide、workflow orchestration。README 强调工具指导、性能等级、25+ 语言，但源码体量较小，更多是包装型工具。
- **sdsrss/code-graph-mcp@960d120**: Rust、Tree-sitter、SQLite/sqlite-vec、BM25+vector RRF、Merkle incremental、context compression、HTTP route tracing、Claude plugin。它的方向更接近“单 binary + hybrid search + route trace”。

可借鉴点：Rust 变体的 Merkle no-change detection、sqlite-vec optional embedding、route tracing、context compression level (L0/L1/L2/L3) 值得单项评估。

不应照搬：README 中很多节省数字来自小型自测和项目自述，缺少 OmniWeave 这种 agent A/B 边界验证；不能作为战略证据。

4.11 Graphify 与 Understand Anything：可视化学习层，而不是核心图引擎

Graphify 与 Understand Anything 的共同方向是“把项目变成知识图/交互 dashboard/报告/导览”。它们对 onboarding、项目展示、非工程用户理解有价值，但不是低延迟 agent retrieval substrate。

Graphify 会生成 `graph.html`、`GRAPH\_REPORT.md`、`graph.json`，覆盖代码、docs、PDF、图片、视频，并提供各 agent 的 skill 安装。Understand Anything 用多 agent pipeline、interactive dashboard、domain view、guided tours、persona-adaptive UI。

OmniWeave 可以借鉴：

- 独立导出 `architecture report/graph report`，作为人读产物。
- 给 onboarding 做可视化，不进入 MCP 默认路径。
- 把“surprising connections/suggested questions”作为 offline report，而不是实时工具噪声。

4.12 SCIP/Sourcegraph：精确索引的事实格式，不是查询引擎本身

4.12.1 定位

SCIP 是 Sourcegraph 推出的 code intelligence protocol。它不是数据库，也不是 agent 工具，而是 indexer 与 consumer 之间的 transmission format。

官方 README 说明它用于 Go to definition、Find references、Find implementations；`scip.proto` 说明 indexer 可处于 compiler precision 与 heuristic syntax analysis 之间。设计文档明确：SCIP 是 transmission format，不是 storage/query format，见 `repos/scip/docs/DESIGN.md:10-16, 63-72`。

4.12.2 关键设计

- Index 推荐 streaming consume，metadata 必须在 stream 开头，减少大 payload 内存压力，见 `repos/scip/scip.proto:20-39`。
- `Document.relative\_path` 规范要求相对 project root、不能 / 开头、不能符号链接、路径分隔符统一，见 `repos/scip/scip.proto:75-91`。
- Symbol string grammar 包含 scheme、package manager/name/version、descriptor，支持跨 repo/-package 唯一性，见 `repos/scip/scip.proto:148-190`。
- 设计文档反对直接编码全图 adjacency list，理由是并行性、内存和 consumer 侧负担，见 `repos/scip/docs/DESIGN.md:97-123`。

4.12.3 OmniWeave 可借鉴

1. **SCIP importer**, 而非 **SCIP core**。OmniWeave 可读取 `index.scip`, 导入 precise same-language references/inheritance, 标注 `provenance=scip`。
2. **Symbol ID 规范**。OmniWeave 当前 ID 偏本地 DB; 未来 snapshot/bundle/export 可参考 SCIP 的 package-aware symbol scheme。
3. **Streaming artifact**。如果做团队共享索引, 不要直接 dump 全图 JSON; 应支持 streaming、version、root path、position encoding。

4.13 Kythe/Glean/CodeQL/Joern/Semgrep/OpenGrep: 重型工业系统的启示

这些系统没有全部本地 clone 深读, 本报告基于官方文档/公开资料定位:

- **Kythe**: 强调跨语言 semantic code graph、facts/edges、serving tables、cross-reference。适合大规模公司代码导航, 不适合轻量 agent MCP core。
- **Glean**: Meta 开源的代码索引系统, 使用 Angle query language, 目标是多语言代码索引与查询。它代表“Datalog-like facts + query”的工业路线。
- **CodeQL**: GitHub 的 semantic code analysis engine, 把代码作为数据库查询, 主要用于 variant analysis/security。强在安全分析, 不在低延迟 agent 上下文。
- **Joern**: Code Property Graph, AST/CFG/PDG 等统一为 CPG, 用 Scala DSL 挖漏洞模式。适合安全研究, 不适合作为日常 agent exploration 默认层。
- **Semgrep/OpenGrep**: 规则/模式匹配、SAST、taint/reachability。适合“找安全/规范模式”, 不是 repo understanding graph。

结论: OmniWeave 应借鉴它们的事实模型纪律、**query safety**、**provenance**、版本化 **schema**、可解释规则, 而不是照搬重型查询语言或安全扫描平台。

4.14 RepoGraph 与 CodeRAG-Bench: 研究评测提醒我们别只测检索

RepoGraph 把 repo-level code graph 加入 SWE-bench/Agentless/SWE-agent。源码显示它大量借鉴 Aider repo-map、Agentless、grep-ast; `construct\_graph.py` 中甚至有对 Python 代码做 `exec(import_statement)` 来识别标准库函数的逻辑, 风险很高, 不适合作为生产参考。

CodeRAG-Bench 关注“检索增强是否提升代码生成”。它的 retrieval 脚本覆盖 BM25、SBERT、OpenAI/Voyage embeddings、repoeval、swe-bench repo retrieval, generation 侧再评估模型输出。这对 OmniWeave 的启发不是实现, 而是评测: 只测 Recall@K 或 token saving 不够, 还要测最终 agent task 是否更快、更稳、更少误读。

5 横向比较

5.1 事实来源: Tree-sitter、LSP、SCIP、embedding 的信任边界

来源	强项	弱点	OmniWeave 应用
Tree-sitter/AST	零配置、快、跨语言、可本地运行	类型/动态分派/宏/导入解析有限	核心 extractor, 配 provenance/confidence。
LSP	同语言精确 references/rename/diagnostics	需要环境、慢启动、跨语言弱	optional validation/edit companion。
SCIP	编译器/indexer 级别 precise facts, 可跨 repo symbol	依赖 indexer、生成成本高, 不是查询引擎	importer/snapshot fact source。

来源	强项	弱点	OmniWeave 应用
Embedding/vector	概念召回强，自然语言入口	不产生结构真相，可解释弱，需状态管理	optional semantic seed。
规则/CPG/CodeQL	安全/variant analysis 强	规则语言重、学习成本高 安全插件/专项 pass，不进默认路径。	

5.2 存储与查询

- **SQLite** 是 **OmniWeave** 当前最优核心。codegraph、OmniWeave、code-review-graph、Codebase-Memory、sdsrss/code-graph-mcp 都选择 SQLite 或 SQLite 衍生能力。它足够本地、可携带、低运维。
- **Tantivy/FTS/vector** 是 **sidecar** 候选。Codanna 的 Tantivy exact/ngram 值得学；sqlite-vec/FastEmbed 适合 optional semantic layer。
- **Neo4j/Falkor/Kuzu** 多后端不适合核心。CGC/Blarify 的多后端路线会带来 schema parity、query translation、deployment、错误处理复杂度。
- 自由 **query language** 必须先安全化。Codebase-Memory 的 SQLite authorizer 是底线；没有 authorizer、timeout、row cap、explain/complexity gate，就不应给 agent 自由查询。

5.3 增量同步

生态里主要有五种：

1. 文件 **watcher + debounce**：codegraph/OmniWeave、semantic-search-mcp；适合开发态，但要处理平台 watcher 差异和 stale state。
2. **Git diff/status**：Codebase-Memory、code-review-graph；适合 review/CI，但对未跟踪文件不完整。
3. **Merkle/hash tree**：sdsrss/code-graph-mcp；no-change detection 快，适合大 repo。
4. **snapshot artifact**：Codebase-Memory；适合团队共享和冷启动。
5. **remote/cloud collection sync**：Claude Context；状态复杂度最高，必须有 lock/recovery。

OmniWeave 当前 watcher + staleness banner 是合理核心；下一步最值得加的是 **hash-verified snapshot import/export**，不是云同步。

5.4 MCP 工具面

项目	工具面趋势	对 agent 的风险
codegraph	默认只暴露 primary explore ，其他 opt-in	低，模型选择成本小。
OmniWeave	默认五个核心工具，强调 explore/node	可继续收缩或按 query shape routing。
Codebase-Memory	14 tools，含 query/architecture/ADR/runtime traces	能力强但选择成本高。
CGC	indexing、watch、Cypher、visualize、bundle、framework tools	极易 tool drift 和输出面不一致。
Serena	tool set/mode 可控，读写工具分层	值得借鉴 mode control。

项目	工具面趋势	对 agent 的风险
code-review-graph	review/change vertical tools 很完整	场景清晰时有效，通用时偏重。

结论：OmniWeave 的主线应是 少工具、强输出、明确 **fallback**。每增加一个 tool，必须证明它减少了 agent 的实际工具调用或错误选择。

6 OmniWeave 的优势：不是“大而全”，而是边界可信

6.1 优势一：抓住 LSP/grep 共同盲区

grep 的盲区是“关系”；LSP 的盲区是“跨语言/跨进程/工作流/动态框架约定”。大部分竞品只解决其中之一：

- Aider 解决“哪些文件值得读”，不是“谁调用谁”。
- Serena/LSP 解决同语言 symbol navigation，不解决 Python→R、Snakemake→tool。
- Claude Context 解决“概念上相关”，不解决 structure。
- CodeQL/Joern 解决安全程序分析，不解决 agent 日常上下文形态。

OmniWeave 的 crossLang、S4 dispatch、workflow DAG、external tool nodes 正好打在这些交集盲区。这个定位应坚持。

6.2 优势二：诚实边界比高召回更稀缺

很多项目把“能猜到”当成“能连边”。OmniWeave 当前更克制：

- 动态路径里 residual `\{...\}\$` 就 skip；
- interpreter 必须处在真实 subprocess call 里，不能只是 echo 字符串；
- fan-out 有 cap；
- heuristic edge 带 confidence；
- static dispatch 只建声明骨架，不伪造运行时 target。

这正是 agent substrate 的信任基础。Agent 会把结构边当成事实来行动；错边会导致错误修改、错误影响面、错误审查结论。OmniWeave 的“宁可漏，不可骗”是优势，不是保守。

6.3 优势三：A/B 证明了真实价值边界

README 当前最有力量的不是数字本身，而是承认边界：correctness 经常 tie；single-point lookup 不赢；concept search 不是 OmniWeave；cross-process at scale 由于运行时路径构造也会 tie。很多竞品还停在“我减少了多少 token”的自述阶段，OmniWeave 已经进入“在哪些 query shape 赢、在哪些不赢”的阶段。

这会带来一个战略好处：产品路线可以少走弯路。比如：

- 不应该把资源投向“让 symbol lookup 比 grep 更快很多”；
- 应该投向 reverse/blast-radius、workflow/crossLang、weak-model guardrail、output sufficiency；
- 不应该把 semantic search 写成核心卖点；
- 可以把 semantic search 作为 seed layer 服务 explore。

6.4 优势四：本地、轻量、可分发

Codebase-Memory、Claude Context、CGC、Blarify 都有更重的状态面：外部 vector DB、多 DB backend、SCIP/Docker/LSP、cloud collection、Neo4j/Falkor。OmniWeave 的核心依赖更少：Node 22.5+、WASM tree-sitter、SQLite。这是安装、CI、隐私、团队采用的实际优势。

但要注意：轻量不是简陋。轻量必须和 artifact、version、integrity、watch freshness、schema migration 一起做，否则只是在本地临时可用。

7 OmniWeave 应该借鉴什么

7.1 优先级 P0：输出面与工具路由继续压实

借鉴对象	做法	OmniWeave 落地建议
codegraph	默认只暴露 primary tool，输出预算按 repo/task 调整	对默认 tools 做 A/B：explore+node+search 是否足够；其余 opt-in。
Aider	PageRank + mention personalization	explore 候选排序引入 query/current-file/recent-symbol personalization。
code-review-graph	context savings 元数据与诚实 benchmark 叙事	MCP 响应可附 compact savings/debug metadata，仅在 verbose 或 eval mode 展示。
Codanna	symbol_id:N 消歧协议	node/explore 输出 stable continuation key，后续工具直接消费。

7.2 优先级 P1：snapshot/artifact 共享

借鉴 Codebase-Memory 的 artifact 方向，但做得更克制：

- omniweave snapshot export：输出 graph.db.zst 或 bundle directory；
- manifest 包含 schema version、OmniWeave version、source root hash、file count、edge kind count、createdAt、platform；
- import 默认 read-only，路径 root 必须显式 remap；
- 不信任 artifact 内任意路径；所有 source snippet 读取仍受 project root guard；
- snapshot import 只加速冷启动，不替代本地 reindex 的真相。

7.3 优先级 P1：可选 SCIP importer

SCIP 是最合理的 precise same-language facts 来源。建议：

1. 首期只支持读取 index.scip，不负责运行 indexer。
2. 只导入 definition/reference/implementation/inheritance 等清晰事实，标注 provenance=scip。
3. 与 Tree-sitter 边冲突时不覆盖，保留 parallel provenance 或 higher confidence。
4. 输出中对 SCIP facts 标注 compiler-backed/third-party indexed。

这样可以补 OmniWeave 在 same-language 精度上的短板，但不会让安装路径变重。

7.4 优先级 P2: semantic sidecar

借鉴 Claude Context、semantic-search-mcp、Cursor 的方向，但不做默认依赖：

- semantic sidecar 只返回 candidate file/symbol IDs；
- explore 用这些 IDs 进入结构图；
- 语义层可以支持 local embedding、sqlite-vec 或外部 provider，但均为 plugin；
- 任何 semantic score 不能变成 structural edge confidence。

7.5 优先级 P2: review vertical skill

借鉴 code-review-graph 的场景化：

- /omniweave:review-delta: 自动取 git diff changed files → impact → affected workflows/tools → test gaps；
- /omniweave:review-pr: base branch diff → risk summary；
- 输出必须短、稳定、可粘贴，不暴露整个 graph JSON；
- 所有 review 指标都要标明 graph-derived 上界还是 git co-change evidence。

8 OmniWeave 不应该借鉴什么

8.1 不要默认追求大工具面

Codebase-Memory 的 14 tools、CGC 的大工具面、code-review-graph 的 vertical tools 都各有道理，但 OmniWeave 的默认 MCP surface 必须克制。对 agent 来说，多一个工具不是多一个能力，而是多一个误选路径。

判断标准：

如果一个新增 tool 不能在 A/B 中降低工具调用、降低 token、减少 fallback read、或提升稳定性，就不应默认暴露。

8.2 不要引入多数据库后端

Neo4j、FalkorDB、Kuzu、Ladybug、remote graph DB 都可以做 enterprise story，但会破坏 OmniWeave 的 local-first 和零配置优势。SQLite 已足够支持当前核心；更值得做的是 schema、index、query plan、snapshot，而不是 backend abstraction。

8.3 不要把 LLM 文档生成塞进核心

Blarify、Graphify、Understand Anything 都做 documentation/report/tour。这些适合 offline product layer，不适合 indexer core。LLM 生成内容不是事实，必须和 graph facts 分开存储、分开 provenance。

8.4 不要把语义搜索当结构事实

向量检索可以帮助发现入口，但它不能回答调用关系、影响半径、workflow artifact、S4 dispatch。语义层若进入核心边表，会破坏 graph trust。

8.5 不要复制安全分析平台

CodeQL/Joern/Semgrep/OpenGrep 很强，但它们解决的是 variant analysis、CPG mining、taint/rule matching。OmniWeave 可在未来以 plugin 接入安全 pass，但不应把 query DSL 或 taint engine 变成 MCP 默认能力。

9 建议路线图

9.1 Phase A: 输出与工具面硬化

1. 为 explore/node/callers/impact 建 query-shape goldens: single-point、reverse、flow survey、file read replacement、crossLang、workflow。
2. 每个工具输出记录: 字符数、source snippets 完整率、omitted reason、fallback read rate。
3. A/B 默认工具面: 5 tools vs 3 tools vs explore-only, 指标为工具调用、token、turns、正确率、模型误选率。
4. 引入 continuation key: 一个 symbol/file/flow 被返回后, 后续工具可用 key 直接定位。

Done when: 默认工具面比当前不增加 correctness 风险, 同时降低平均 tool mis-pick 或 fallback read。

9.2 Phase B: snapshot artifact

1. 定义 omniweave-snapshot.json manifest。
2. 支持 export/import read-only snapshot。
3. 引入 schema version and compatibility check。
4. 做 corrupt snapshot、wrong root、stale source、malicious path regression tests。

Done when: 干净 checkout 可在无全量 reindex 的情况下读 snapshot, 但任何源码变更都能触发 stale warning/reindex path。

9.3 Phase C: SCIP importer

1. 读取 SCIP protobuf, 导入 definitions/references。
2. 映射到 OmniWeave nodes/edges, 保留 original symbol string。
3. 输出中展示 provenance=scip。
4. 与 Tree-sitter edge 冲突时不静默覆盖。

Done when: 在 TS/Java/Rust fixture 上 same-language references 明显更精确, 且不影响 cross-Lang/workflow/S4 gates。

9.4 Phase D: semantic sidecar

1. 先做 internal seed, 不暴露为默认 MCP tool。
2. 本地 provider 优先, 外部 provider 明确 opt-in。
3. Eval 只测“是否帮助 concept query 找到正确 explore seed”, 不把它混入 structural correctness。

Done when: concept query 的 first useful file rate 提升, 且 structure query 无 regression。

9.5 Phase E: review vertical skill

1. 用现有 impact/workflow/tool edges 生成 PR review context。
2. 输出 risk/test gaps/affected tools/affected workflows。
3. 区分 graph-derived upper bound 和 git co-change evidence。
4. 不默认安装, 作为 skill/plugin 分发。

10 最终判断

OmniWeave 应该避免两个诱惑：一个是向上变成“通用知识图/可视化平台”，一个是向下变成“更快 grep/更多语言 parser”。前者会拖垮核心，后者会失去差异化。
最强路径是：

把 **OmniWeave** 做成 **coding agent** 的可信结构控制层：少工具、强输出、边界可信、跨语言/跨进程/**workflow/dynamic-dispatch** 优先、本地可分发，并用 **A/B** 证明每个新增能力真的减少 **agent** 成本。

这也是与所有竞品拉开距离的地方。Codebase-Memory 更像“代码库记忆平台”；Serena 更像“IDE/LSP agent 工具层”；Claude Context 更像“语义召回层”；CodeQL/Joern 是“安全分析层”；Graphify/Understand Anything 是“学习/可视化层”。OmniWeave 的最佳位置不是吞并它们，而是成为它们之下或旁边的可信结构骨架。

A 本地源码快照

项目	commit	文件数	体量 (KB)	备注
codegraph	a893156	387	20076	OmniWeave 直接基座。
codebase-memory	684e35b	1676	1280540	C/Tree-sitter/SQLite/MCP，含 vendored grammars。
cgc	c235df7	998	116640	多后端、多工具、SCIP optional。
serena	dd7eb6d	906	12104	LSP/IDE 工具层。
aider	5dc9490	691	159884	repo-map/ranking/context compression。
codanna-clean	cb2c095	822	15800	Rust/Tantivy/vector/symbol-id。
claude-context	627eb2b	177	35092	semantic search + Milvus/Zilliz。
blarify	ada9bbd	400	14220	Tree-sitter/LSP/SCIP/graph DB/LLM docs。
code-review-graph	b72413c	287	15520	PR review vertical。
code-graph-mcp	374d41e	30	868	Python MCP graph variant。
code-graph-mcp-2	960d120	224	5540	Rust MCP graph variant。
semantic-search-mcp	40a1b01	48	976	local semantic search baseline。
scip	75d1151	219	3728	protocol repo。
graphify	a4d09ae	592	14144	report/visual graph layer。
understand-anything	7f5a717	420	51304	plugin/dashboard/learning layer。
repograph	6c3977d	209	7416	research graph for SWE-bench。
coderag-bench	f9e100ca	27674	1244536	retrieval/generation benchmark。

B 关键证据索引

证据	用途
OmniWeave/README.md:35-98	OmniWeave agent A/B 边界、wins/ties/losses。
OmniWeave/README.md:135-164	crossLang、S4、workflow DAG、external tool graph。
OmniWeave/CHECKPOINT.md:38-55	数据流与 skip-over-guess 边界规则。
OmniWeave/src/resolution/callback-synthesizer.ts:1941-2039	general cross-language edge gates。
codegraph/src/mcp/tools.ts	primary explore/output budget/default tool surface。
codegraph/src/extraction/index.ts	parse worker、timeout、recycle、watch integration。
codebase-memory/src/mcp/mcp.c:280-448	14 MCP tools 与平台化工具面。
codebase-memory/src/store/store.c:540-581	SQLite authorizer/security。
codebase-memory/src/pipeline/pipeline.c	RAM-first multi-pass pipeline。
cgc/src/codegraphcontext/tools/indexing/pipeline.py	async indexing + framework post-passes。
serena/src/serena/tools/symbol_tools.py	LSP symbol/find/reference tools。
aider/aider/repomap.py	PageRank/personalization/token budget repo-map。
codanna-clean/src/retrieve.rs:66-190	symbol_id:N 消歧 workflow。
codanna-clean/src/indexing/pipeline/mod.rs:260-543	DISCOVER/READ/PARSE/COLLECT/INDEX pipeline。
claude-context/packages/mcp/src/handlers.ts:10-154	indexing task cancel/snapshot poisoning recovery。

证据	用途
blarify/blarify/ code_references/ hybrid_resolver. py:20–185	SCIP/LSP hybrid resolver。
code-review-graph/ docs/REPRODUCING. md:10–60, 100–128	token estimate calibration/eval reproducibility。
scip/scip.proto: 20–39, 75–190	streaming index/document/symbol schema。
scip/docs/DESIGN. md:10–16, 97–123	SCIP 是 transmission format，避免全图 adjacency。

C 外部资料与链接

References

[1] colbymchenry/codegraph. <https://github.com/colbymchenry/codegraph>

[2] DeusData/codebase-memory-mcp. <https://github.com/DeusData/codebase-memory-mcp>

[3] Codebase-Memory paper, arXiv:2603.27277. <https://arxiv.org/abs/2603.27277>

[4] oraios/serena. <https://github.com/oraios/serena>

[5] Aider-AI/aider. <https://github.com/Aider-AI/aider>

[6] bartolli/codanna. <https://github.com/bartolli/codanna>

[7] zilliztech/claude-context. <https://github.com/zilliztech/claude-context>

[8] blarApp/blarify. <https://github.com/blarApp/blarify>

[9] tirth8205/code-review-graph. <https://github.com/tirth8205/code-review-graph>

[10] SCIP repository. <https://github.com/scip-code/scip>

[11] Sourcegraph, SCIP announcement. <https://sourcegraph.com/blog/announcing-scip>

[12] Sourcegraph code intelligence docs. https://docs.sourcegraph.com/user/code_intelligence

[13] Kythe storage documentation. <https://kythe.io/docs/kythe-storage.html>

[14] Meta Engineering, Glean open source code indexing. <https://engineering.fb.com/2024/12/19/developer-tools/glean-open-source-code-indexing/>

[15] GitHub CodeQL overview. <https://codeql.github.com/docs/codeql-overview/about-codeql/>

[16] Joern Code Property Graph docs. <https://docs.joern.io/code-property-graph/>

[17] Semgrep documentation. <https://docs.semgrep.dev/>

- [18] OpenGrep repository. <https://github.com/opengrep/opengrep>
- [19] Cursor secure codebase indexing. <https://cursor.com/blog/secure-codebase-indexing>
- [20] Cursor semantic and agentic search docs. <https://cursor.com/docs/agent/tools/search>
- [21] GitHub Copilot repository indexing docs. <https://docs.github.com/en/copilot/concepts/context/repository-indexing>
- [22] OpenAI Codex Agent Skills docs. <https://developers.openai.com/codex/skills>
- [23] OpenAI Codex repository. <https://github.com/openai/codex>
- [24] RepoGraph. <https://github.com/ozyyshr/RepoGraph>
- [25] CodeRAG-Bench. <https://github.com/code-rag-bench/code-rag-bench>